

1) Why data cleaning is needed

In real projects, the data you get is rarely perfectly ready for model training. Common issues include:

- Unclear or wrong column names (e.g., `Unnamed: 0`)
- Inconsistent labeling (e.g., some `condition` values are uppercase, others lowercase)
- Noisy text (e.g., `\r\n`, HTML entities like `'`)

This section shows how to fix these problems using `datasets`.

2) What “slicing and dicing” means

It means doing practical dataset operations such as:

- Inspecting small parts of the data
- Filtering out bad rows
- Renaming columns
- Transforming values in columns

Hugging Face Datasets offers many operations similar to Pandas, but on `Dataset` and `DatasetDict` objects.

3) Which dataset is used

They use the Drug Review Dataset (from the UC Irvine ML Repository). It contains:

- `drugName`: name of the drug
- `condition`: the condition/disease being treated
- `review`: patient-written review text
- `rating`: satisfaction rating (1 to 10)
- `date`: review date
- `usefulCount`: how many people found the review useful
- `Unnamed: 0`: a suspicious column that looks like an ID

4) Downloading and unzipping the data

They download a zip file and extract it:

- `wget` downloads the file

- `unzip` extracts it

After extraction, the `.tsv` files are available locally.

5) Loading TSV files

TSV means “Tab Separated Values”. It is like CSV, but uses a tab character instead of a comma.

So they still use the `"csv"` loader, but specify the delimiter:

```
from datasets import load_dataset
```

```
data_files = {"train": "drugsComTrain_raw.tsv", "test": "drugsComTest_raw.tsv"}  
drug_dataset = load_dataset("csv", data_files=data_files, delimiter="\t")
```

Meaning:

- `data_files` defines which file is train and which is test
- `delimiter="\t"` tells the loader the separator is a tab
- `drug_dataset` becomes a `DatasetDict` containing `train` and `test`

6) Why take a small random sample

Before any serious analysis, it's good to look at a small random sample so you quickly understand:

- what the columns look like
- what kinds of values exist
- what problems are present

They do:

```
drug_sample = drug_dataset["train"].shuffle(seed=42).select(range(1000))  
drug_sample[:3]
```

Explanation:

`shuffle(seed=42)`

- Randomly shuffles the dataset
- Seed makes it reproducible (same shuffle each time)

`select(range(1000))`

- `select()` expects indices
- `range(1000)` means pick the first 1000 items after shuffling

`drug_sample[:3]`

- Displays the first 3 examples from that sample

7) Issues found from the sample

From the sample output, they notice three main quirks:

Issue 1: **Unnamed: 0** looks like an ID

It looks like an anonymized identifier for each patient/row. This often happens when a dataset was saved with an index column but without a proper name.

Issue 2: **condition** has inconsistent casing

Examples:

- `ibromyalgia` (lowercase)
- `Inflammatory Conditions` (capitalized)

If you don't fix this, statistics and modeling can become inconsistent (same condition counted as different labels).

Issue 3: **review** text contains noise

They see:

- `\r\n` (line breaks / carriage returns)
- `'` (HTML entity; commonly represents an apostrophe)

This noise should be cleaned before NLP preprocessing.

8) Verifying that **Unnamed: 0** is a unique ID

Hypothesis: **Unnamed: 0** is a unique patient ID.

They test it with `Dataset.unique()`:

```
for split in drug_dataset.keys():
    assert len(drug_dataset[split]) == len(drug_dataset[split].unique("Unnamed: 0"))
```

Meaning:

- `len(drug_dataset[split])` = number of rows
- `unique("Unnamed: 0")` = unique values in that column
- If counts match, then every row has a unique value in that column

`assert` ensures the condition is true; otherwise it raises an error.

9) Renaming the column

`Unnamed: 0` is not meaningful, so they rename it to `patient_id`:

```
drug_dataset = drug_dataset.rename_column(
    original_column_name="Unnamed: 0", new_column_name="patient_id"
)
```

`rename_column()` applies across both train and test splits (because `drug_dataset` is a `DatasetDict`).

After this, features become:

`patient_id, drugName, condition, review, rating, date, usefulCount`

10) “Try it out” exercise

They suggest practicing with `unique()` to compute:

- number of unique drugs in train/test
- number of unique conditions in train/test

Conceptually:

- `len(drug_dataset["train"].unique("drugName"))`
- `len(drug_dataset["train"].unique("condition"))`

And similarly for test.

11) Normalizing `condition` labels with `map()`

Goal: make all conditions lowercase so labels become consistent.

They define:

```
def lowercase_condition(example):  
    return {"condition": example["condition"].lower()}
```

This takes one example (row) and returns an updated `condition`.

Then they run:

```
drug_dataset.map(lowercase_condition)
```

12) Why the error occurs (`NoneType`)

They get:

```
AttributeError: 'NoneType' object has no attribute 'lower'
```

Meaning:

- some rows have `condition = None`
- `None` is not a string
- so calling `.lower()` fails

So there are missing values in the dataset.

13) Removing rows where `condition` is `None` (filter)

Solution: drop rows where `condition` is missing.

`filter()` keeps rows for which the function returns `True`.

They could write:

```
def filter_nones(x):  
    return x["condition"] is not None
```

But they use a lambda instead:

```
drug_dataset = drug_dataset.filter(lambda x: x["condition"] is not None)
```

Meaning:

- keep the row only if `condition` is not None

14) What lambda means

A `lambda` is a small anonymous function in Python:

Format:

`lambda arguments: expression`

Examples:

- `lambda x: x * x`
- `(lambda x: x * x)(3)` returns 9
- `(lambda base, height: 0.5 * base * height)(4, 8)` returns 16.0

Here it's used because the filter logic is short and only needed once.

15) Running lowercase again after filtering

After removing None rows, `.lower()` will work:

```
drug_dataset = drug_dataset.map(lowercase_condition)
```

Then they verify:

```
drug_dataset["train"]["condition"][:3]
```

Example output shows lowercase conditions, confirming it worked.

16) Meaning of the final sentence

“It works! Now that we’ve cleaned up the labels, let’s take a look at cleaning up the reviews themselves.”

Meaning:

- condition labels are now cleaned and consistent

- next step is cleaning the `review` text (handling `\r\n`, HTML entities, etc.)
- that review-cleaning part has not started yet in this excerpt